

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

**Дисциплина:**

*“Программирование”*

## **ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2**

Вариант 5-22

**Выполнила:**

Студентка гр. 3148 Жук Анастасия



**Проверил:**

Волков А. Г.

Санкт-Петербург  
2023г.

## Задание

Назовем характеристикой кольца сумму его элементов. Сдвинуть два первых найденных кольца порядка  $K$  с одинаковыми характеристиками на  $V$  элементов по часовой стрелке.

Тип элементов: short.

## MakeFile

```
1 .PHONY: all clean
2
3 APP=lab2avzN3148
4 CFLAGS=-Wall -Wextra -Werror -g
5
6 all: $(APP)
7
8 $(APP):
9     gcc -o $(APP) $(CFLAGS) $(APP).c
10
11 clean:
12     rm $(APP)
13 |
```

## Примеры работы программы на различных исходных данных

```
user@user-HP-Laptop-15s-eq3xxx:~$ make all
gcc -o lab2avzN3148 -Wall -Wextra -Werror -g lab2avzN3148.c
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 -m 3
Ошибка: отсутствует обязательный аргумент.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 -k 3
Ошибка: '-k' не является числом.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 k 3
Ошибка: 'k' не является числом.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 5 k
Ошибка: 'k' не является числом.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 cat dog
Ошибка: 'cat' не является числом.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 5 5
Исходная матрица:
-10551 -27344 14267 -15736 -2015
15222 18367 1222 27490 24095
-27216 30576 -32322 18988 11186
-4006 -5841 30214 25799 -18707
25398 15034 -27083 -30120 -7709
Введите порядок кольца и количество сдвигов:
3 3
Кольца с одинаковыми характеристиками не найдены.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 -m 3 3
Введите строки матрицы:
-1 2 3
2 -4 5
2 -3 1
Исходная матрица:
-1 2 3
2 -4 5
2 -3 1
Введите порядок кольца и количество сдвигов:
2 3
Результат:
2 -4 3
-1 5 1
2 2 -3
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 -k 3 3
Ошибка: опция '-k' не поддерживается.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 -m cat 3
Ошибка: 'cat' не является числом.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 -m 3 dog
Ошибка: 'dog' не является числом.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 -m cat dog
Ошибка: 'cat' не является числом.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 -m 3 3
Введите строки матрицы:
1 2 3
Ошибка: строка не должна начинаться с пробела!
```

```
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 -m 3 3
Введите строки матрицы:
1 2 3
Ошибка: некорректный ввод данных - слишком много пробелов!
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 -m 3 3
Введите строки матрицы:
cat 1 2
Ошибка: некорректный ввод данных - присутствуют не только цифры!
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 -m 3 3
Введите строки матрицы:
11111111 1111 11
Ошибка: тип данных некорректный!
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 -m 3 3
Введите строки матрицы:
1 2 3 4 5
Ошибка: много переменных!
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab2avzN3148 -m 3 3
Введите строки матрицы:
1 2
Ошибка: недостаточно переменных!
user@user-HP-Laptop-15s-eq3xxx:~$
```

## Исходный текст программы с комментариями

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <time.h> //Чтобы использовать функцию time()
#include <string.h> //Для NULL
#include <ctype.h> //Для isdigit

short* smeshenie(short arr[], int i1, int j1, int m, int k, int v) {
    short new[k * k - (k - 2) * (k - 2) + v];
    //создаем массив, который будет представлять из себя кольцо со сдвигами
    int i = i1, j = j1;
    //заполняем его сдвинутыми элементами
    for (int p = v; p < k * k - (k - 2) * (k - 2) + v; p++) {

        if (p < k * k - (k - 2) * (k - 2)) new[p] = arr[i * m + j];
        else new[p - (k * k - (k - 2) * (k - 2))] = arr[i * m + j];

        if (i == i1 && j < j1 + k - 1) j++;
        else if (j == j1 + k - 1 && i < i1 + k - 1) i++;
        else if (i == i1 + k - 1 && j > j1) j--;
        else if (j == j1 && i > i1) i--;
    }

    //а теперь этот новый массив нужно вставить на место старого кольца
    i = i1;
    j = j1;
    for (int p = 0; p < k * k - (k - 2) * (k - 2); p++) {
        arr[i * m + j] = new[p];
        if (i == i1 && j < j1 + k - 1) j++;
        else if (j == j1 + k - 1 && i < i1 + k - 1) i++;
        else if (i == i1 + k - 1 && j > j1) j--;
        else if (j == j1 && i > i1) i--;
    }
    return arr; //вернет массив со сдвинутым кольцом
}

int main(int argc, char *argv[]) {
    // Проверка запуска с переменной среды, включающей отладочный вывод.
    // Пример запуска с установкой переменной LAB2DEBUG в 1:
    // $ LAB2DEBUG=1 ./lab2avzN3148 5 5
    char *DEBUG = getenv("LAB2DEBUG");
    if (DEBUG) {
        fprintf(stderr, "Включен вывод отладочных сообщений\n");
    }

    int n = 0, m = 0;
    srand(time(NULL)); //Для генерации случайных чисел
    //считываем n и m, если они есть
    if (argc < 3) {
        printf("Ошибка: недостаточно аргументов.\n");
        return 0;
    }
    else if (argc == 3) {
        if (strcmp(argv[1], "-m") == 0) {
```

```

        printf("Ошибка: отсутствует обязательный аргумент.\n");
        return 0;
    }
    else if (sscanf(argv[1], "%d", &n) == 0) {
        printf("Ошибка: \"%s\" не является числом.\n", argv[1]);
        return 0;
    }
    else if (sscanf(argv[2], "%d", &m) == 0) {
        printf("Ошибка: \"%s\" не является числом.\n", argv[2]);
        return 0;
    }
}
else if (argc == 4) {
    if (strcmp(argv[1], "-m") != 0) {
        printf("Ошибка: опция '%s' не поддерживается.\n", argv[1]);
        return 0;
    }
    else if (sscanf(argv[2], "%d", &n) == 0) {
        printf("Ошибка: \"%s\" не является числом.\n", argv[2]);
        return 0;
    }
    else if (sscanf(argv[3], "%d", &m) == 0) {
        printf("Ошибка: \"%s\" не является числом.\n", argv[3]);
        return 0;
    }
}

//если корректно введена размерность матрицы, то можно ее создавать
if (n != 0 && m != 0) {
    //рандомные значения в матрице
    short arr[n * m];
    if (argc == 3) {
        printf("Исходная матрица:\n");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                short w = rand();
                arr[i * m + j] = w;
                printf("%hd ", w);
            }
            printf("\n");
        }
    }

    //пользователь сам вводит значения
    else if (argc == 4) {
        printf("Введите строки матрицы:\n");
        int i = 0, j = 0; //координаты вводимого значения
        int g = 0; //шаг
        short s; //считанное значение
        long long t;
        int p; //сдвиг
        char str[7 * m + 1]; //6 * m + (m - 1) + m + 1
        while (i < n) {
            //считываем строку, вытаскиваем из нее подходящие числа и обрабатываем
            fgets(str, 7 * m, stdin);
            if (str[0] == ' ' && str[0] != '-') {
                printf("Ошибка: строка не должна начинаться с пробела!\n");
                return 0;
            }
        }
    }
}

```

ошибки

```

    }
    if (str[0] == '\n') {
        printf("Ошибка: недостаточно переменных!\n");
        return 0;
    }
    while (str[g] != '\0' || g < 7 * m + 1) {
        if (isdigit(str[g]) || str[g] == '-') {
            sscanf(str + g, "%lld%n", &t, &p);
            if (t > 32767 || t < -32768) {
                printf("Ошибка: тип данных некорректный!\n");
                return 0;
            }
            else s = t;
            if (str[g + p] != ' ' && str[g + p] != '\n' && str[g + p] != '-') {
                printf("Ошибка: некорректный ввод данных - 
присутствуют не только цифры!\n");
                return 0;
            }
            arr[i * m + j] = s;
            j++;
            if (j > m) {
                printf("Ошибка: много переменных!\n");
                return 0;
            }
            if (str[g + p] == '\n') break;
            g += p + 1;
        }
        else if (str[g] == ' '){
            printf("Ошибка: некорректный ввод данных - слишком 
много пробелов!\n");
            return 0;
        }
        else {
            printf("Ошибка: некорректный ввод данных - присутствуют 
не только цифры!\n");
            return 0;
        }
    }
    i++;
    if (j < m) {
        printf("Ошибка: недостаточно переменных!\n");
        return 0;
    }
    j = 0;
    p = 0;
    g = 0;
}
printf("Исходная матрица:\n");

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        printf("%d ", arr[i * m + j]);
    }
    printf("\n");
}

printf("Введите порядок кольца и количество сдвигов:\n");

```

```

int k, v;
scanf("%d %d", &k, &v);

//массив ans нужен для того, чтобы считать характеристики колец
int l = 65536 * (k * k - (k - 2) * (k - 2)); //такая размерность нужна для того, чтобы мы могли
обращаться к массиву по суммам, а не бегать по нему и искать схожие значения сумм
int ans[l]; // (power(2, 8 * sizeof(short)) * (k * k - (k - 2) * (k - 2)) = 65536 * (k * k - (k - 2) * (k -
2))

for (int i = 0; i < l; i++)
    ans[i] = -1; //забили -1 для того, чтобы дальнейшая проверка прошла корректно
int flag = 0;
int x1 = 0, x2 = 0; //начальные координаты колец
for (int i = 0; i <= n - k; i++) {
    for (int j = 0; j <= m - k; j++) {
        int a = 0;
        a += arr[i * m + j] + arr[(i + k - 1) * m + j] + arr[i * m + (j + k - 1)] + arr[(i + k - 1) * m +
(j + k - 1)];

        for (int b = 1; b < k - 1; b++) {
            a += arr[(i + b) * m + j] + arr[i * m + (j + b)] + arr[(i + k - 1 - b) * m + (j + k -
1)] + arr[(i + k - 1) * m + (j + k - 1 - b)];
        } //подсчитываем характеристику кольца
        if (ans[a + 32768 * (k * k - (k - 2) * (k - 2))] == -1) {
            ans[a + 32768 * (k * k - (k - 2) * (k - 2))] = i * m + j;
        }
        else {
            x1 = ans[a + 32768 * (k * k - (k - 2) * (k - 2))];
            x2 = m * i + j;
            flag = 1;
            break;
        }
    }
    if (flag == 1) break;
}

if (x1 == x2) { //пара не нашлась
    printf("Кольца с одинаковыми характеристиками не найдены.\n");
    return 0;
}

int i1 = x1 / m, j1 = x1 - i1 * m;
int i2 = x2 / m, j2 = x2 - i2 * m;

smeshenie(arr, i1, j1, m, k, v);
smeshenie(arr, i2, j2, m, k, v);

printf("Результат:\n");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        printf("%d ", arr[i * m + j]);
    }
    printf("\n");
}

return EXIT_SUCCESS;
}

```